Sentra

AI-powered sentiment analysis for customer feedback



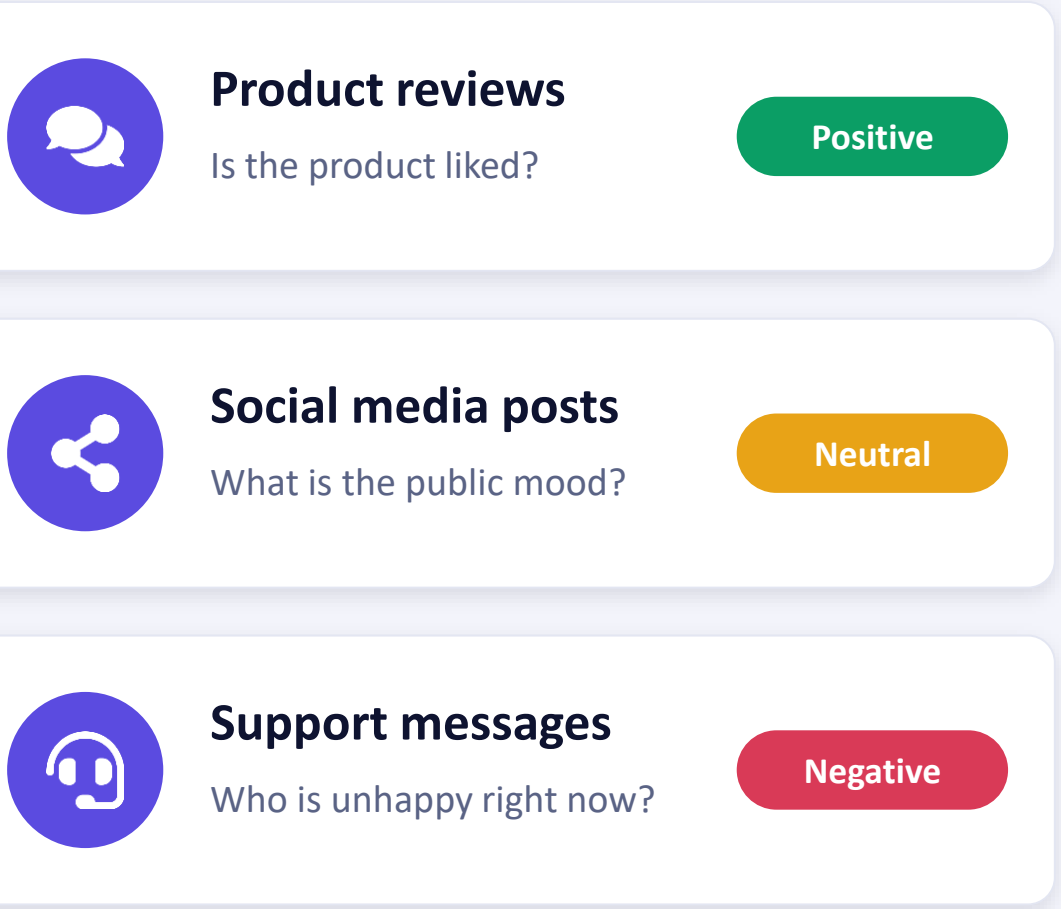
Anushay Ayat | AI & Data Science, SMIT

Full-stack project: Python, machine learning, Firebase, Vercel

Feedback is too big to read by hand

Businesses collect thousands of reviews, posts and messages every week. Reading each one is slow, and two people rarely judge the same text the same way.

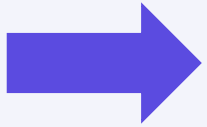
Sentiment analysis turns every text into one clear label, so teams can see what customers feel at a glance.



One sentence in, one verdict out

Input (illustrative example)

“The delivery was fast and the quality is amazing.”



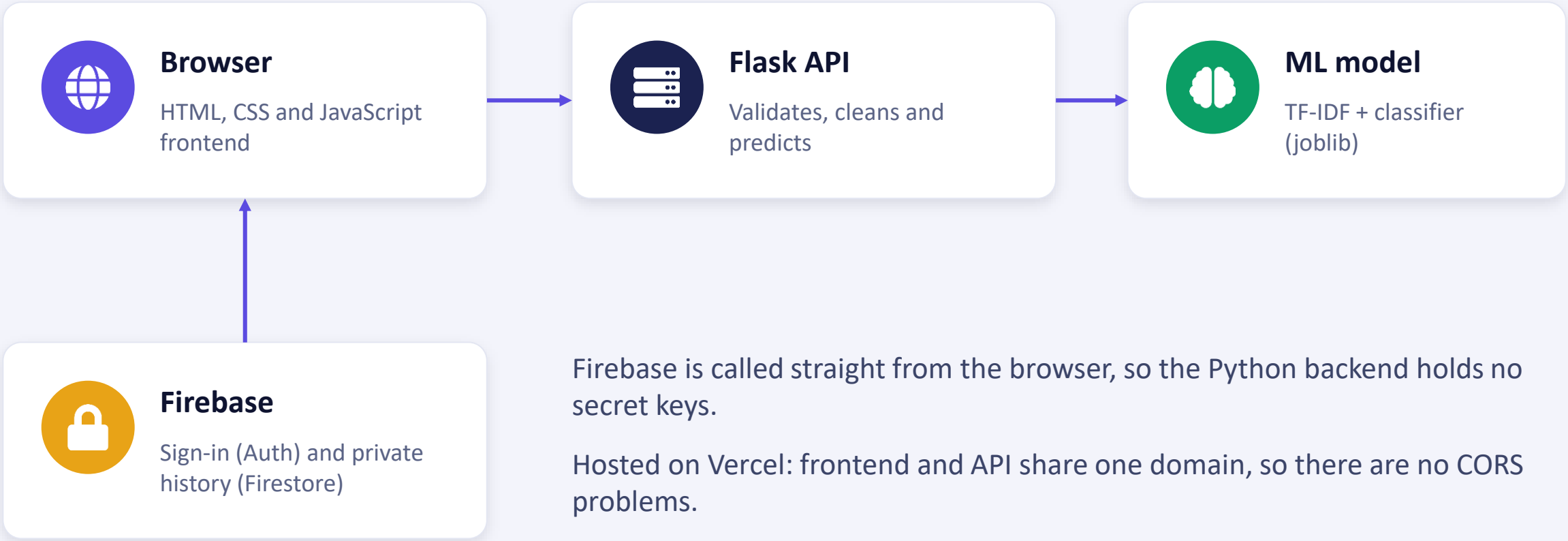
Positive — Predicted

Neutral

Negative

Sentra also shows the confidence for each class and highlights the words that influenced the decision.

How the pieces fit together



Firebase is called straight from the browser, so the Python backend holds no secret keys.

Hosted on Vercel: frontend and API share one domain, so there are no CORS problems.

From raw text to a trained model



1. Dataset

TweetEval sentiment with fixed splits



2. Clean

Lowercase, links, mentions; keep “not”



3. TF-IDF

Word and word-pair features



4. Classify

Logistic Regression or Naive Bayes



5. Evaluate

Held-out test set, once

Vectorizer and model are fitted on training data only, so test results are honest.

Dataset and text preparation

Dataset

TweetEval: Sentiment

SemEval-2017 Task 4, published by Cardiff NLP on Hugging Face

Three classes: Negative, Neutral, Positive

Ready-made train, validation and test splits (about 45k, 2k and 12k texts)

Cleaning rules

- Lowercase and normalise characters
- Replace links and @mentions with tokens
- Collapse stretched letters (soooo to soo)
- Keep stop words so “not good” stays negative
- Same function in training and prediction

Choosing and measuring the model

Candidates

- Logistic Regression with five regularisation strengths
- Complement Naive Bayes with two smoothing values
- Best candidate is picked on the validation set (macro F1)
- Final scores come from the untouched test set

Metrics are macro-averaged, so the harder Neutral class counts as much as the others.

Accuracy

58.8%

share of correct predictions

Precision

57.8%

of predicted X, how many are X

Recall

59.3%

of actual X, how many were found

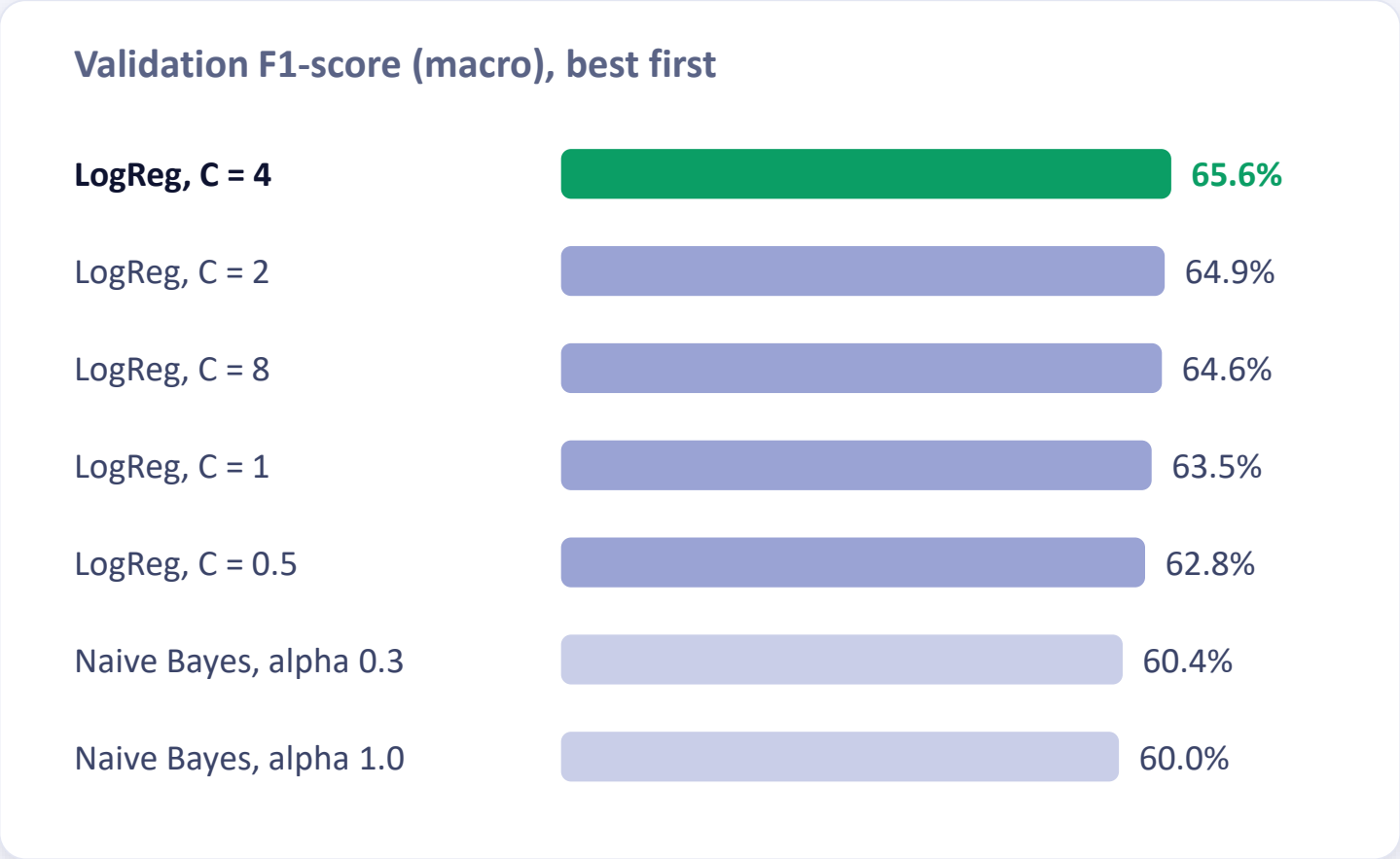
F1-score

58.4%

balance of precision and recall

Macro-averaged on 12,284 held-out test texts

Logistic Regression won the comparison



Validation vs test

65.6%

F1 on validation texts, used to pick the model

58.4%

F1 on the untouched test texts: the honest estimate

A small, clear Python API

POST `/api/predict`

Text in, sentiment out

GET `/api/model-info`

Dataset and test metrics

GET `/api/health`

Is the model loaded?

Input must be 3 to 1000 characters with letters. Every error is JSON with a code and a message (400, 404, 405, 413, 503, 500).

Response


```
{
  "sentiment": "Positive",
  "confidence": 0.87,
  "scores": {
    "Negative": 0.04,
    "Neutral": 0.09,
    "Positive": 0.87
  },
  "keywords": [ { "word": "amazing",
    "sentiment": "Positive" } ]
}
```

A frontend built like a real product

Explainable results

Influential words are highlighted in your text

Live dashboard

Totals, distribution donut and a 7-day chart

History tools

Search, filter, delete and export to CSV

Fully responsive

Desktop, tablet and phone layouts

Light and dark

Theme toggle that remembers your choice

Clear feedback

Loading states and plain-language errors

Accounts and private history



Authentication

Email/password and Google sign-in handled by Firebase



Cloud Firestore

Each analysis is saved under the signed-in user



Security rules

Users can read, create and delete only their own records

`users / {uid} / predictions / {id}`

text, sentiment, confidence, scores, createdAt. Entries are validated and cannot be edited. No secret keys exist in the repository.

How it was tested

17

automated tests pass

Python unit tests for cleaning and for every API response and error

- Positive, Neutral and Negative inputs
- Empty, too short, too long and numbers-only text
- Malformed JSON and unknown routes
- Missing model returns a clean 503
- Browser run: desktop and phone width, no layout overflow
- Sign-up, login, history, filter, search and delete flows
- Final check on the live Vercel link

From laptop to a live link

1



Push to GitHub

Commit code, the trained model and the README

2



Deploy on Vercel

Import the repo; vercel.json serves the frontend and Flask API

3



Authorise the domain

Add the Vercel URL in Firebase authorised domains

Check /api/health on the live site: it should report that the model is loaded.

Limits and next steps

Honest limits

- Trained on English tweets, so long reviews can score lower
- Sarcasm and mixed opinions are hard for a bag-of-words model
- Neutral overlaps with both other classes

Next steps

- Train on product-review data to match real use
- Try a transformer model for better accuracy
- Batch upload: analyse a whole CSV of reviews
- Support more languages, including Urdu

Thank you

Questions are welcome

LIVE APP

sentra-ai-powered-sentiment-analysi.vercel.app

GITHUB

github.com/scientistayat-engineer/Sentra-AI-Powered-Sentiment-Analysis

